Fire Detection & Autonomous Response at the Edge

D7065E Embedded Intelligence at the Edge, Architecture Document

Group 3: Binay Mishra

Contents

1. Summary **2**

2. Use case and building context **2**

3. Requirements **3**

4. Architecture **4**
4.1 Context diagram (C4 Level 1) 4
4.2 Container diagram (C4 Level 2) 5
4.3 Component diagram (C4 Level 3) 6
4.4 Design decisions and alternatives 8

5. Interfaces and data contracts **9**

6. Simulating the sensor values (the physical model) **10**

7. Autonomous services and data pipeline **10**
7.1 The autonomous services 10
7.2 The data pipeline 11

8. Behaviour **12**
8.1 Key scenario (dynamic diagram) 12
8.2 Door lifecycle (state diagram) 15

9. Deployment and component placement **16**

10. Test plan **17**

11. Evaluation and results **17**

12. Dashboard **20**

13. Risks and critical reflection **22**

1. Summary

This system detects fires in the A-building at LTU and responds autonomously. It is built in two tiers. **Tier 1** is an edge controller per floor, running on a Raspberry Pi: it reads every room's smoke, CO and temperature, applies a deterministic multi-sensor rule gated by an isolation forest, and releases its own floor's fire doors within one 5 s tick, with no dependency on a server, a broker or a language model. **Tier 2** is a server agent: it asks a self-hosted LLM for a verdict on every edge alert, coordinates the other floors, computes evacuation routes, drives the 3D viewer, and stands the building down when readings are clear. A standby controller on a second node takes over a floor within 30 s if its active controller dies; the server takes over a floor if both controllers are gone; and the whole response still works when the LLM is unreachable.

The building, its 950 rooms on three floors, 124 fire doors, sprinklers and 3D view, is provided by BuildSim. This project provides the physics, the sensor degradation, the two tiers of intelligence and the response around it. The guiding principle is **life-safety actions at the edge, judgement at the server, graceful degradation everywhere**.

Measured on the homelab cluster during the acceptance run of 3–4 October 2026: ignition to own-floor doors released in 14.7 s; isolation-forest inference p50 17.8 ms on the Pi; LLM verdict p50 2.1 s, p95 3.2 s; verdict to cross-floor release 25 ms; standby promotion 30 s after controller loss. Nine acceptance scenarios pass, including controller failover mid-incident, server takeover of a floor with no controller, LLM outage, broker restart and cold start.

Field	Value
Project title	Fire Detection & Autonomous Response at the Edge
Team members	Binay Mishra (Group 3, solo)
Use case	Autonomous fire detection, floor-local door release, building-wide coordination and evacuation
Repository	https://github.com/applied-ai-ltu/d7065e
Version / date	v2.0 / 2026-10-12

Changes since the proposal. Three design changes were made because the evidence demanded them.

1. **From one agent to two tiers.** The proposal had a single server agent polling BuildSim and deciding everything, with the LLM on the critical path. Measured LLM latency (5–7 s per plan) and the realisation that door release must not depend on a server made the case for a per-floor edge controller that acts first and reports second. The LLM moved off the life-safety path and now only confirms, rejects and coordinates.
2. **The LLM returns a verdict, not a plan.** The proposal had the LLM generate the response plan (sprinklers, doors, routes). The plan is now derived deterministically from a verdict `{is_fire, confidence}`; sprinklers are mechanical in the physics model and never commanded. This halved the verdict latency (5.2 s to 2.1 s p50) and removed a class of unsafe plans.
3. **Control moved out of Grafana.** The Business Forms plugin proved unsuitable for button control; fire control is a small FastAPI service and Grafana is monitoring only. Scope grew from one room to the whole building once the gateway was parameterised by floor.

2. Use case and building context

The building. The A-building at LTU is modelled in BuildSim across three floors: level0 (318 rooms, 41 fire doors), level1 (351 rooms, 46 fire doors) and level2 (287 rooms, 37 fire doors). BuildSim is the shared state of the cyber-physical system: it owns the building model, exposes

a REST API to read and write equipment state, provides a navigation graph per floor with cross-floor stairwell edges, and streams live state to a 3D web viewer. BuildSim is not modified; it is instrumented. That seam, the building behind BuildSim's API on one side and this project's processes on the other, is what would let the same software drive a real building.

How the system connects to BuildSim. Three things talk to BuildSim, and nothing else touches the building state directly:

- The **physics engine and sensor gateway** (inside the server process) read the floor plans, the door graph and the equipment list at start-up, advance the room physics, degrade the readings with realistic faults, and publish them over MQTT. They publish 956 spaces, the 950 rooms plus six combined rooms and corridors that BuildSim lists as equipment but not as rooms.
- Each **edge controller** discovers its floor's fire doors from the equipment list and sets their state directly (`PUT /api/actuators/{id}/state`). This is the only life-safety write path and it does not pass through the server.
- The **server agent** pushes visualisation (effects, alert cards, coverage) and, only for a floor whose controllers are all gone, sets that floor's doors itself.

Actors. The **building operator** watches the 3D viewer and the Grafana dashboards and may start or stop test fires, trigger false alarms and clear the viewer through the control API. **Building occupants** are modelled by the occupancy simulator and are the reason evacuation routes are computed. Neither actor is on the control path; the loop runs autonomously.

The scenario the demo tells. A fire starts in room A109 on level0. Smoke and CO rise and spread through the door graph to A108, A110 and the corridor. Within three ticks the level0 controller sees smoke above 10 %obs/m and CO above 20 ppm, releases all 41 fire doors on its floor and alerts the server. The server asks the LLM, gets a confirmed verdict two seconds later, releases the doors on level1 and level2, publishes evacuation routes and paints the viewer. Neighbouring rooms are assessed in parallel as their alerts arrive. When the fire is out and readings have been clear for a minute, the server stands each room down; when the last one is clear it re-locks every floor and clears the viewer. If the LLM is down the verdict comes from the same rule the edge used; if the level0 controller dies mid-incident its standby takes over 30 s later without touching the doors.

Why it matters. Fire is the highest-stakes building automation scenario: late detection kills, false alarms cost evacuations, and a local fire needs a building-wide response. Three concerns drive the requirements: **act locally and fast**, **judge with context**, and **degrade gracefully**. The first is why detection and door release live on the floor; the second is why an LLM reviews every alert; the third is why every component has a documented failure behaviour that was exercised live.

3. Requirements

Each requirement is a testable statement with a unique ID, traced to a design element (Sections 4–9) and to a verification in the test plan (Section 10). Functional requirements (FR) say what the system does; non-functional requirements (NFR) say how well; safety requirements (SAF) are the ones a fire-safety engineer would insist on.

ID	Type	Requirement	Pri.	Acceptance criterion	Verified by
FR-01	Func.	Detect a fire within 60 s of ignition	Must	Edge `fire` decision logged within 60 s	T-06
FR-02	Func.	Distinguish a real fire from a smoke-only nuisance	Must	Smoke-only event rejected by Tier 2, no door command	T-05, T-07

ID	Type	Requirement	Pri.	Acceptance criterion	Verified by
FR-03	Func.	Release the fire doors of the affected floor without any server involvement	Must	Door release issued in the same tick as the `fire` decision, server offline or not	T-06, T-02
FR-04	Func.	Compute evacuation routes that avoid affected rooms	Must	Route published on confirm; affected rooms not on the path	T-06
FR-05	Func.	Propagate fire to physically adjacent rooms only	Must	Spread follows the door graph and a 30-unit wall radius; no spread across courtyards	T-06, T-U
FR-06	Func.	Stand the building down only on evidence	Must	Stand-down after 60 s of clear readings per room; refused while a room reads critical	T-06, T-U
NFR-01	Non-func.	Monitor all 950 rooms on three floors	Must	All rooms present in the database and in each controller's room count	T-04
NFR-02	Non-func.	Survive LLM unavailability without losing detection or response	Must	Fallback confirms within one tick; cross-floor release still issued	T-02
NFR-03	Non-func.	Ingest the full sensor stream	Must	Pipeline sustains $\geq$400 readings/s with an empty queue	T-09
NFR-04	Non-func.	Run detection on the edge device	Must	ONNX detector on the Pi, p95 $<$ 100 ms per room	T-06, T-09
NFR-05	Non-func.	No secrets or Vault tokens in the repository or the pods	Must	CI lint passes; server reads secrets from a Kubernetes Secret	T-10
NFR-06	Non-func.	Survive a broker restart	Should	Edges and server reconnect; fire handled afterwards	T-03
NFR-07	Non-func.	Tolerate a stuck sensor	Should	Frozen sensor cannot block detection or hold the stand-down	T-03, T-U
SAF-01	Safety	Release fire doors on every floor when a fire is confirmed on any floor	Must	Release commands to all other floors within 1 s of the verdict	T-06
SAF-02	Safety	Keep a floor covered when its active controller dies	Must	Standby promotes within 30 s; doors already released stay released	T-08
SAF-03	Safety	Keep a floor covered when both its controllers are gone	Must	Server takes the floor over within 30 s and actuates its doors directly	T-01
SAF-04	Safety	Doors are never re-locked by a handover, restart or false alarm	Must	Re-lock only on a stand-down with the floor clear, or on ALL CLEAR	T-05, T-07, T-08

A deliberate tension runs through these requirements: detection speed (FR-01, FR-03) pulls toward aggressive thresholds, while false-alarm rejection (FR-02) pulls toward conservative ones. The two tiers resolve it by splitting the decision: the edge errs toward releasing doors, which is cheap and safe, and the server errs toward not escalating across the building, which is expensive, with readings as the tie-breaker.

4. Architecture

4.1 Context diagram (C4 Level 1)

The context view fixes the system boundary. The system reads and commands the building only through BuildSim; the operator watches the viewer and the dashboards and controls through the API; occupants are modelled, not connected; and the LLM inference service and the observability stack are external systems the design must survive without.

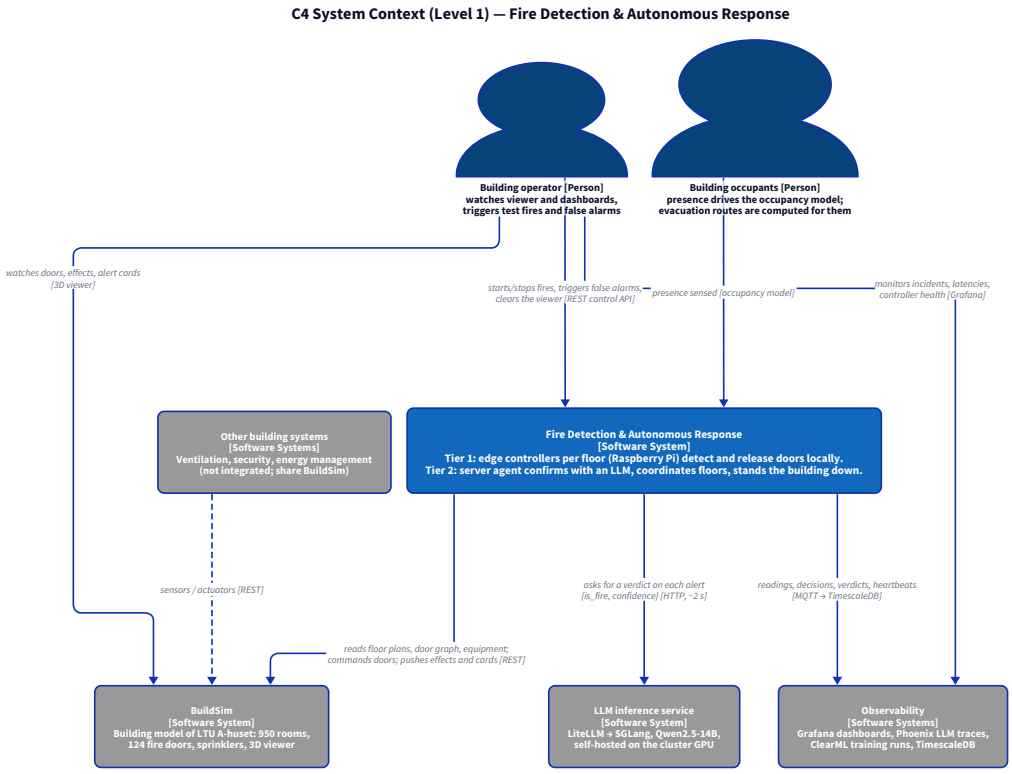


Figure 1: System context. The Raspberry Pi edge controllers are inside the system boundary; the LLM service and observability are outside it and optional for life-safety behaviour.

4.2 Container diagram (C4 Level 2)

Each box in Figure 2 is a process or service, placed on the node where it runs. Tier 1 is six edge controllers: an active per floor on the Raspberry Pi and a standby per floor on an amd64 worker, built from the same image with the same resource spec. Tier 2 is the server process, which hosts the physics engine and sensor gateway, the agent, the MQTT-to-database pipeline and the control API. Mosquitto connects the tiers; readings flow down to the controllers, alerts, decisions and heartbeats flow up to the server, commands flow down again. BuildSim is reached by REST from both tiers; the LLM only from Tier 2.

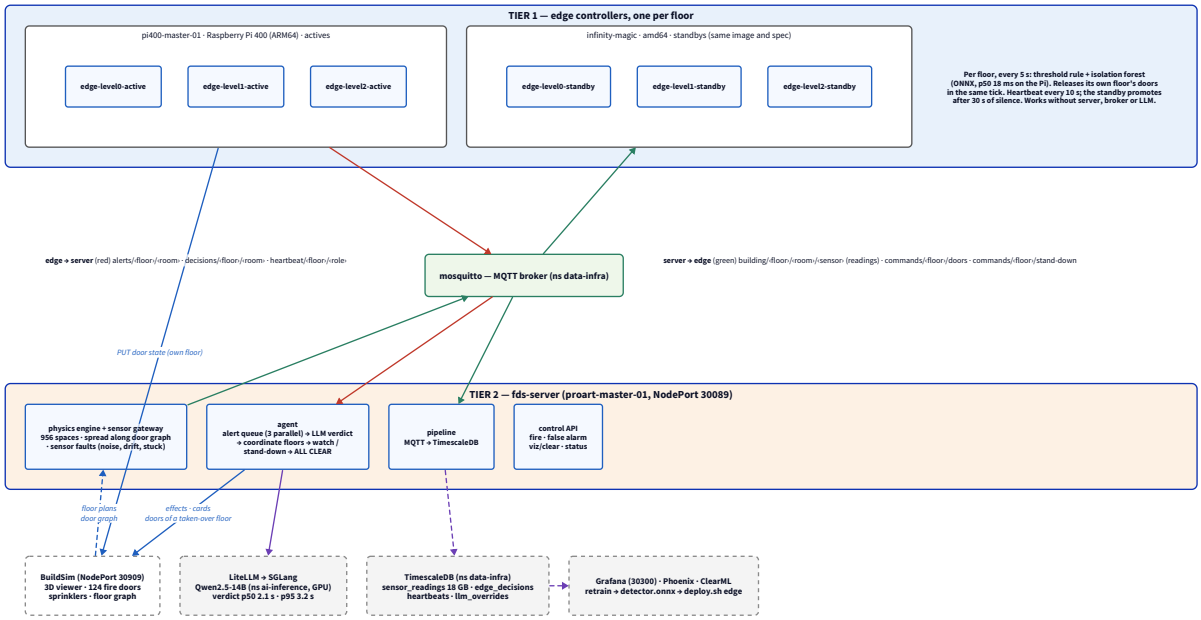


Figure 2: Containers and their placement on the k3s cluster. Red edges carry alerts and heartbeats; green edges carry readings and commands; blue edges are REST calls to BuildSim; purple edges are data and model flows.

The closed loop is: the physics engine advances the rooms and publishes degraded readings; the controller on that floor decides; it writes door state into BuildSim; the engine reads the door graph and door states and the next tick reflects them, as does the viewer. The server sits beside this loop, not inside it.

4.3 Component diagram (C4 Level 3)

The two decision components are the ones worth opening. Figure 3 shows them side by side, as they run: the edge controller's per-room tick on the left and the server agent's per-alert handling on the right, with the MQTT messages that connect them.

Tier 1, per room every 5 s. A deterministic multi-sensor rule comes first: smoke ≥ 10 %obs/m and CO ≥ 20 ppm is a fire, full stop, and the learned model can never veto it. A second rule covers a known-faulty CO sensor (value frozen for 60 s): then strong smoke alone (≥ 20) confirms. Below those, the isolation forest gates the ambiguous band: a room with smoke ≥ 5 that the model flags as anomalous goes to the server as `possible`; a room with smoke ≥ 10 goes to the server even if the model is quiet (safety net). On the first `fire` on its floor the controller releases every fire door it owns and alerts the server; alerts are rate-limited per room (state change, 1.5× rise, or every 60 s) so the server is never flooded.

Tier 2, per alert, three in parallel. Alerts are de-duplicated (same room and severity within 60 s is dropped; a confirmed room is never re-assessed). The agent builds the features the LLM sees: readings and their rates, the list of sensors the edge marked stale, the rooms at risk from spread, and the evacuation route. The LLM returns a verdict. Confidence ≥ 0.7 confirms: cross-floor release (once per incident), routes, viewer effects and a red card. Anything else puts the room on a 60 s watch with an orange card, and readings decide: clear (smoke < 2 and CO < 5 on live sensors) stands the room down, fire-level readings escalate it on the deterministic rule, anything in between keeps watching. A confirmed room is stood down after 60 s of clear readings. When no room is confirmed or watched anywhere, ALL CLEAR re-locks every floor and clears the viewer.

If the LLM is unreachable, the fallback applies the same multi-sensor rule with confidence 0.9 or 0.2.

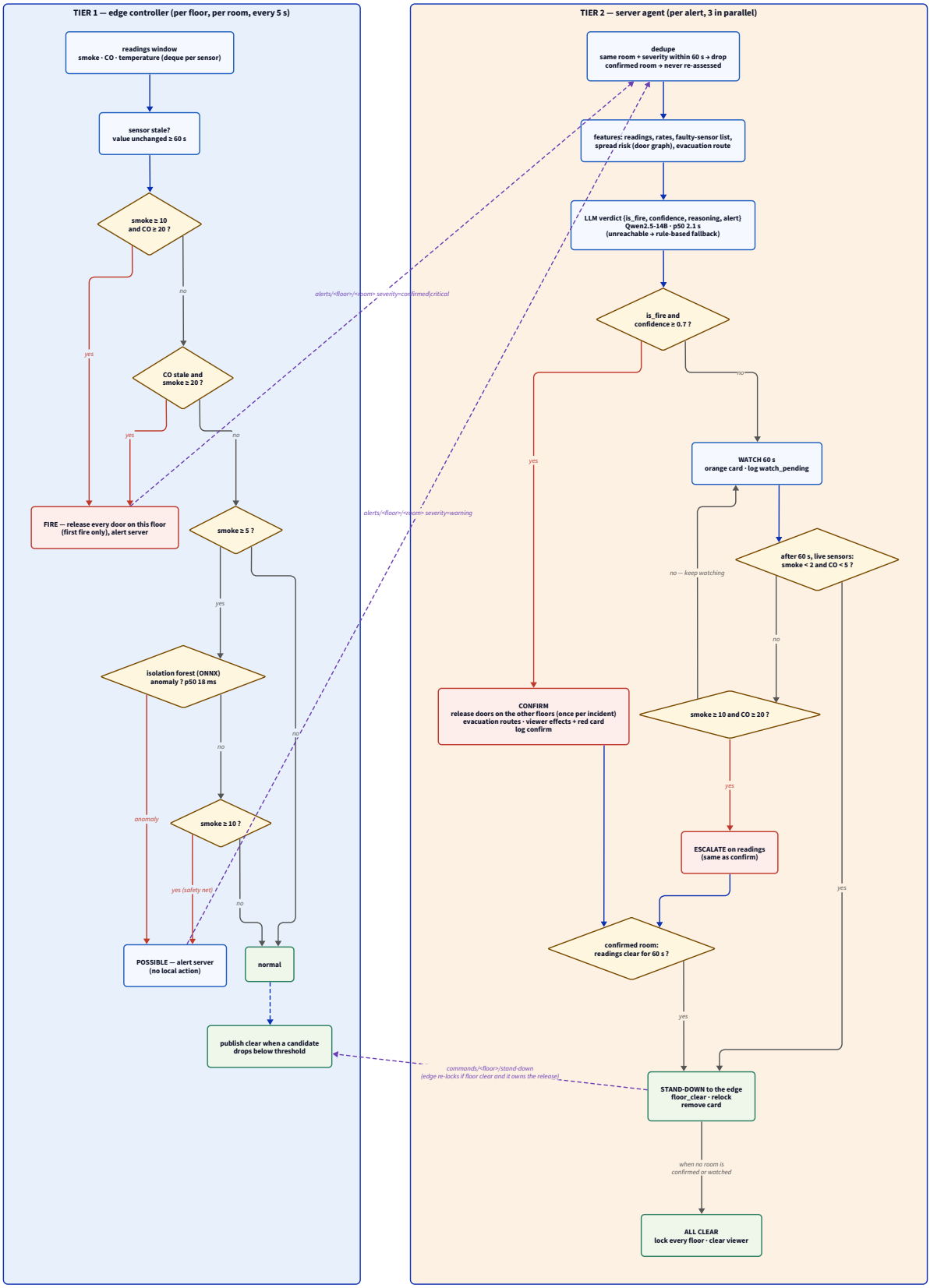


Figure 3: Decision components of both tiers. Red paths act; grey paths defer; the purple dashed edges are the MQTT messages between the tiers.

4.4 Design decisions and alternatives

Each row states the option chosen, the alternative seriously considered, the requirement it serves, and the trade-off accepted.

Decision	Chosen	Alternatives	Justification (requirement)	Trade-off
Topology	Two tiers: edge per floor + server	Server-only; edge-only	Door release must not depend on a server, a broker or an LLM (FR-03, SAF-01–03). Cross-floor coordination, routes and explanation need a building-wide view.	Two code bases, state split across tiers
Edge decision	Deterministic rule; isolation forest gates only the ambiguous band	Model decides; thresholds only	The rule is auditable and never vetoed by a learned model (FR-01). Thresholds alone cannot tell smoke-only from fire (FR-02); the model earns its place on the ambiguous band.	Hand-set thresholds
Edge model	Isolation forest, ONNX Runtime on the Pi	Autoencoder; TFLite; classifier	17.8 ms p50 on a Pi 400, sklearn-native export, parity 1.000, retrainable from real windows (NFR-04). A classifier needs labels the system only gets from the LLM over time.	Less expressive than a neural model
Tier-2 model	Qwen2.5-14B via LiteLLM/SGLang, verdict-only contract	Full-plan generation; cloud API; calibrated decision model	The plan is derived deterministically from `{is_fire, confidence}`; 5.2 s to 2.1 s p50 per verdict; self-hosted so it works offline (NFR-02). Calibrated decision models (Jev and open-weight equivalents) fit the verdict function better but were weeks old at the time; see Section 13.	Confidence is not calibrated
Alert handling	Bounded concurrency (3) + per-room dedupe	Serial queue	Alert→verdict queueing was 24 s p50 under a cluster of alerts; parallel assessment removed it. Dedupe keeps LLM load proportional to incidents, not ticks.	Ordering is per room, not global
Door ownership	Edge owns its floor, server owns cross-floor; stand-down carries `floor_clear` and `relock`	Server owns all doors	Survives failover and restart without state transfer: a fresh controller re-locks because the server tells it the floor is clear, not because it remembers opening (SAF-02, SAF-04).	Two release flags per floor to reason about
Failover	Active/standby per floor, heartbeat 10 s, promote at 30 s; server takeover when both are silent	Single controller; k8s restart only	A pod restart takes 20 s and loses state; a hot standby on another node keeps detecting throughout (SAF-02). Server takeover covers the case where the node itself is gone (SAF-03).	Standby costs a second pod per floor
Stale sensors	Value frozen $\geq$ 60 s is reported stale; stand-down ignores it, detection uses strong smoke alone	Trust every reading	A CO sensor stuck at 5.09 ppm held three floors open; one stuck at 0.00 in the fire room blocked detection (NFR-07). Both seen live.	A frozen but true value is treated as faulty
Spread model	BuildSim door graph + 30-unit wall radius	50-unit radius	A plain radius crossed a courtyard (A109 to A323 at 45 units); the door graph alone is a spanning tree and would starve the spread (FR-05).	Centroid geometry, not walls

Decision	Chosen	Alternatives	Justification (requirement)	Trade-off
Broker / store	Mosquitto / TimescaleDB	Kafka / InfluxDB	Footprint, SQL for Grafana, hypertables for 18 GB of readings (NFR-03).	One more service each
Deployment	k3s, node-local images, `deploy.sh`	ArgoCD GitOps	Images never leave the cluster (`imagePullPolicy: Never`), so GitOps could not deploy them; `selfHeal` would also have reverted the scale-to-zero used in failover tests.	No declarative sync
Secrets	Kubernetes Secret `fds-app`; Vault for tooling	Vault token in the pod	No root token in the namespace (NFR-05); tooling on the dev machine keeps using Vault.	Secret created by hand once

5. Interfaces and data contracts

Two communication patterns are used. **MQTT pub/sub** carries everything time-critical between the tiers because one publisher feeds several consumers, the broker buffers short outages, and topic hierarchy gives per-floor routing for free. **REST** carries every call to BuildSim and the operator's control API, because each is a one-to-one request whose result the caller must confirm.

Every reading is a JSON object with explicit units:

```
{ "sensor_id": "level0-A109-smoke", "room": "A109",
  "level": "level0", "type": "smoke", "unit": "%obs/m",
  "value": 14.29, "timestamp": "2026-10-04T13:46:07Z" }
```

An edge alert carries the snapshot the controller decided on, including the stale-sensor list and the model's cost:

```
{ "room": "A109", "floor": "level0", "decision": "fire",
  "severity": "confirmed", "smoke": 14.29, "co": 37.68,
  "temperature": 17.63, "stale": [], "if_anomaly": true,
  "if_ms": 17.49, "node": "pi400-master-01", "role": "active" }
```

Interface	Protocol	Endpoint / topic	Notes
Gateway → broker	MQTT	`building/{level}/{room}/{type}`	one message per sensor per second, degraded values
Edge ← broker	MQTT	`building/{level}/#`, `commands/{level}/#`, `heartbeat/{level}/#`	a controller subscribes to its own floor only
Edge → broker	MQTT	`alerts/{level}/{room}`, `decisions/{level}/{room}`, `heartbeat/{level}/{role}`	alerts QoS 1; room names with '+' sanitised to '_' in the topic
Server → edge	MQTT	`commands/{level}/doors` {action: release | lock}; `commands/{level}/stand-down` {room, floor_clear, relock}	lock is unconditional for a floor without an own fire
Edge → BuildSim	REST	`PUT /api/actuators/{id}/state` {closed_unlocked | closed_locked}	own-floor doors, 16 concurrent PUTs
Server → BuildSim	REST	`/api/effects`, `/api/alerts`, `/api/room-appearance`, `/api/coverage`; actuators of a taken-over floor	replace semantics: the agent pushes whole lists
Server → LLM	HTTP	LiteLLM chat completion, JSON response `{assessment: {is_fire, confidence, reasoning}, alert: {severity, title, message}}`	timed; `llm_ms` stored per verdict

Interface	Protocol	Endpoint / topic	Notes
Pipeline → TimescaleDB	SQL	`sensor_readings`, `edge_decisions`, `heartbeats`, `llm_overrides`, `fire_events`	async batch inserts
Operator → control API	REST	`POST /fire/start`, `/fire/stop`, `/false-alarm/trigger`, `/viz/clear`; `GET /status`	NodePort 30089
Grafana ← TimescaleDB	SQL	dashboards built by `grafana/build_dashboards.py`	5 s refresh

Door states are BuildSim's own: `closed_locked` is the normal state; `closed_unlocked` is released. A heartbeat carries node, role and whether the controller is acting; a standby's heartbeat with `acting=true` is how the dashboard shows a promotion.

6. Simulating the sensor values (the physical model)

BuildSim provides the rooms but not their physics, so the physics engine generates the readings. The aim is enough structure, trends, propagation and genuine responses to the building state, that both tiers react to meaningful data and the loop closes.

Temperature follows a discrete heat balance per room with envelope loss to the outside, fire heat proportional to intensity, occupant heat from the occupancy model, and a sprinkler term. Sprinklers are mechanical: a head activates when the room temperature passes its activation point and then reduces the fire term. They are not commanded by either tier, which matches real buildings and keeps the control path to the doors.

Smoke and CO ramp with fire intensity and decay exponentially when the fire is out. CO decays slower than smoke: after a 45 s fire the fire room's CO halves roughly every minute, which is why ALL CLEAR follows the stop by about seven minutes. CO does *not* rise during a nuisance event, which is the signal both tiers use to tell a fire from burnt toast.

Propagation. Each tick, a burning room pushes heat, smoke and CO to its neighbours with a factor $1 - d/60$ of the centroid distance d. Neighbours are rooms joined by an edge of BuildSim's door graph plus rooms within 30 units; the previous 50-unit radius made A323 a neighbour of A109 across a courtyard, so every A109 fire "jumped" to the far wing. Cross-floor spread uses BuildSim's 32 stairwell links at an effective distance of 80.

Occupancy drives the baseline heat load with per-room-type daily profiles and sampled head-counts.

Sensor degradation. The gateway degrades the ground truth with four fault modes: Gaussian noise, response lag, slow drift, and **stuck** sensors that freeze at a value and recover randomly. The stuck mode turned out to be the most consequential: during acceptance a CO sensor froze at 5.09 ppm, just above the stand-down threshold, and another froze at 0.00 in the fire room. Both are now handled (Section 7.1), and both are the kind of fault a real building produces.

7. Autonomous services and data pipeline

7.1 The autonomous services

Tier 1: the edge controller (`edge/edge_controller.py`) runs as one process per floor and role. It keeps a short window of readings per sensor per room, tracks when each sensor's value last changed, and every 5 s evaluates each candidate room as in Figure 3. The isolation forest runs through ONNX Runtime on a six-feature vector (smoke, CO, temperature and their rates);

on the Pi 400 this costs 17.8 ms p50 and 33 ms p95 per room. On the first `fire` on its floor the controller releases all of its doors with 16 concurrent PUTs, then alerts the server. It publishes a `clear` decision when a candidate drops back below threshold so the server's view never freezes on a stale value. It accepts three commands: `release` and `lock` for the doors, and `stand-down` for a room, which it refuses if that room still reads critical. It re-locks only when the floor is clear and it owns the release, or when the server says `relock` because the controller is new. It skips a door state it already wrote in this process. The standby runs the same code on the same readings with `acting=false`; after 30 s without the active's heartbeat it promotes, reports a `failover`, and demotes when the heartbeat returns.

Tier 2: the server agent (`src/fds/agent/agent.py`) consumes alerts from a queue with three in flight. Its policy is the right-hand side of Figure 3. Two details matter for safety. First, a doubted alert never undoes the edge: the floor's doors stay released until readings decide. Second, every stand-down carries what the server knows that the controller might not: whether the floor is clear and whether to re-lock. The agent also keeps the per-floor heartbeat table; a floor silent for 30 s (90 s after the server's own boot) is taken over: the server subscribes to that floor's readings, applies the threshold rule itself, and actuates its doors directly. The first heartbeat hands the floor back.

Why two techniques, and why this split. The isolation forest learns the boundary of normal from real operation and runs anywhere; the LLM reads context, including the faulty-sensor list, spread risk and the route, and explains itself. Neither is trusted alone: the rule beats the model on the edge, and readings beat the LLM on the server. The LLM's confidence is not a calibrated probability (Section 13), which is why it only gates escalation and stand-down, and why its rejection of an edge-confirmed room is a watch, not a reversal.

Degradation. LLM unreachable: the fallback applies the multi-sensor rule and the strong-smoke-with-stale-CO rule, with confidence 0.9 or 0.2, and the cards say "fallback mode". Broker restart: paho reconnects; the controller keeps ticking meanwhile and retries its initial connect with backoff. Server restart: the controllers keep their floors; the next stand-down carries `relock`. Controller restart or node loss: Section 8.

Retraining (`fds.models.retrain`, a ClearML task) rebuilds the detector from real operation: per-room 5 s windows from the readings table, the windows the LLM rejected (weighted $\times 20$), and augmented nuisance events, fitted with contamination 0.02, evaluated against real normal rooms, LLM-confirmed fires, held-out nuisance windows and the deployed model, exported to ONNX and checked for sklearn/ONNX parity. The first retrain on real data replaced a synthetic-trained model that flagged 97 % of real normal rooms; the retrain after the acceptance weekend reproduced the deployed model (Section 11), which is the behaviour one wants from a stable loop.

7.2 The data pipeline

Everything the system does leaves a row. The pipeline consumer subscribes to `building/#`, `decisions/#`, `alerts/#` and `heartbeat/#` and writes to five hypertables: `sensor_readings` (183 M rows, 18 GB after eight days), `edge_decisions` (every Tier-1 decision, alert and door event with `if_ms` and `if_anomaly`), `heartbeats`, `llm_overrides` (every Tier-2 verdict, watch, stand-down, takeover and failover with `llm_ms`) and `fire_events`. Grafana reads them directly; the retraining job reads readings and verdicts, so the LLM's judgements become the labels for the next edge model (Figure 4).

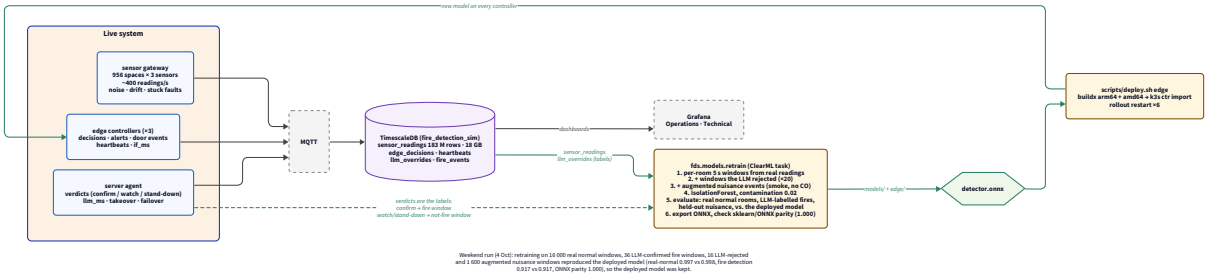


Figure 4: Data pipeline and the retraining loop: readings and verdicts flow into TimescaleDB; the retrain task turns verdicts into labels and ships a new ONNX model to every controller.

Measured ingestion is about 410 readings/s sustained with the consumer queue at zero between bursts; a start-up burst of 974 queued messages drained within the minute.

8. Behaviour

8.1 Key scenario (dynamic diagram)

Figure 5 is the measured baseline fire from the acceptance run (scenario 6): ignition in A109 at 13:45:52.680 UTC; the level0 controller confirms at 13:46:07.357 on smoke 14.3 and CO 37.7 and has released all 41 doors before its log line is written, 14.7 s after ignition and with no server involved; the server receives the alert 1 ms later, gets the verdict after 2 967 ms and releases level1 and level2 25 ms after that. The fire is stopped at 13:46:37.688; the rooms stand down one by one as their readings clear for 60 s, and ALL CLEAR re-locks all three floors at 13:53:59.171.

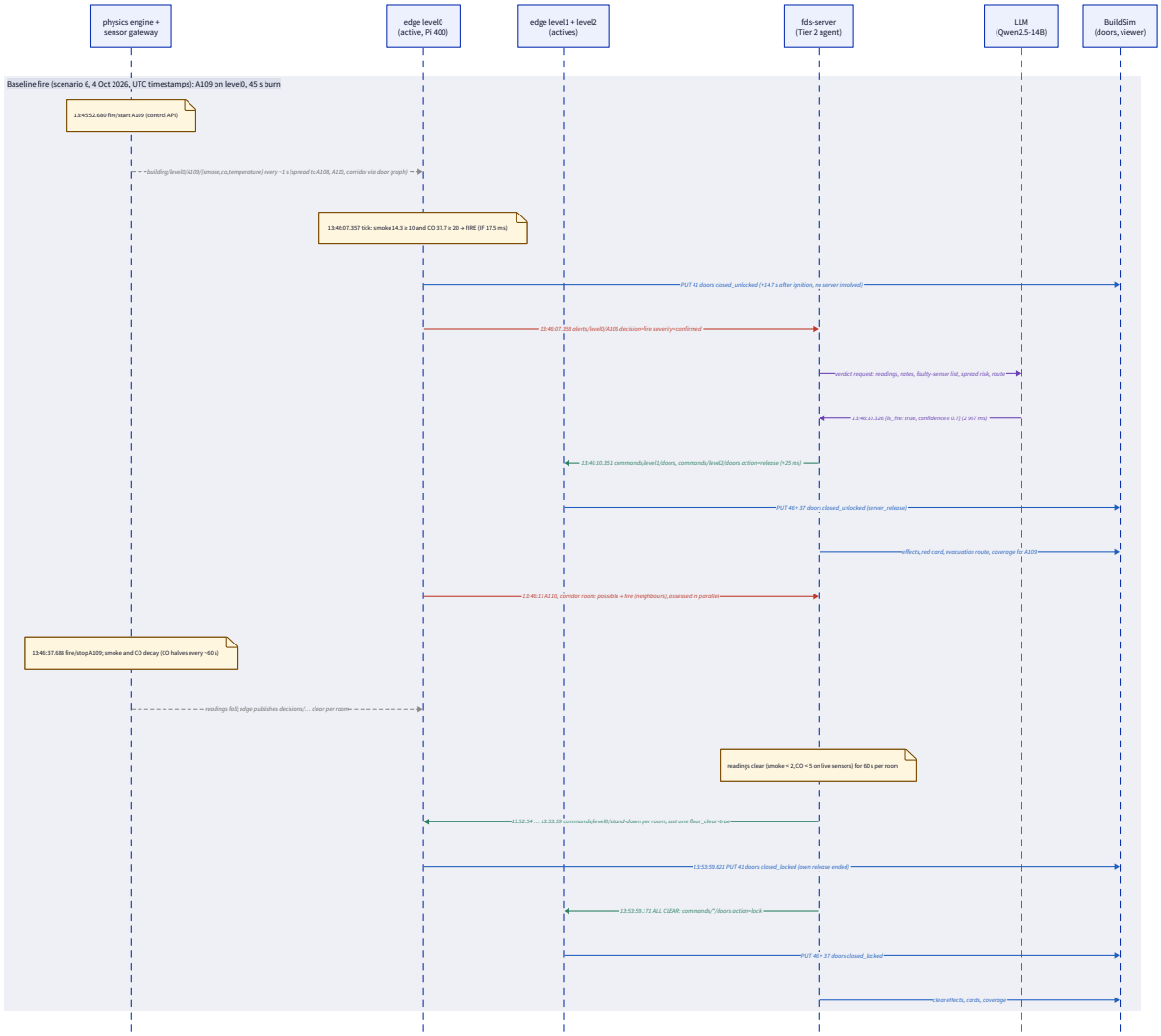


Figure 5: Baseline fire, scenario 6, with measured timestamps. The edge acts in the first 15 s; the server adds the building-wide response three seconds later.

Two further sequences were measured. In **failover mid-incident** (Figure 6, scenario 8) the level1 active pod was deleted while its doors were released: the standby promoted 30 s after the last heartbeat, reported the failover, kept the doors as they were, and demoted 5 s later when the Deployment's replacement pod sent its first heartbeat. The replacement had no memory of the incident; the stand-down's `relock=true` made it lock anyway. In **server takeover** (Figure 7, scenario 1) both level2 controllers were scaled to zero: the server took the floor over 30 s later, released its 37 doors directly when a level0 fire was confirmed, and handed back 20 s after the controllers returned.

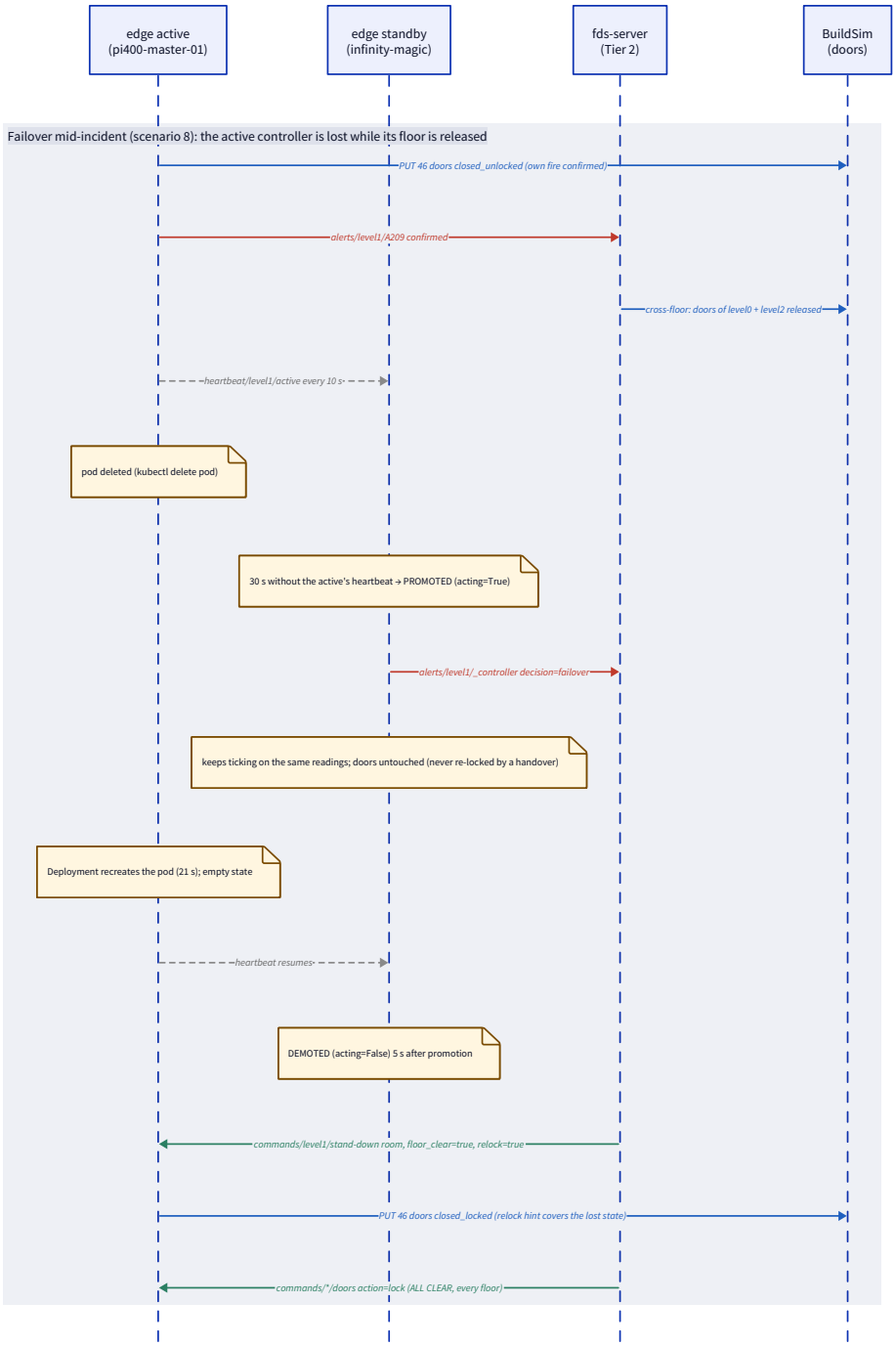


Figure 6: Scenario 8: active controller lost mid-incident.

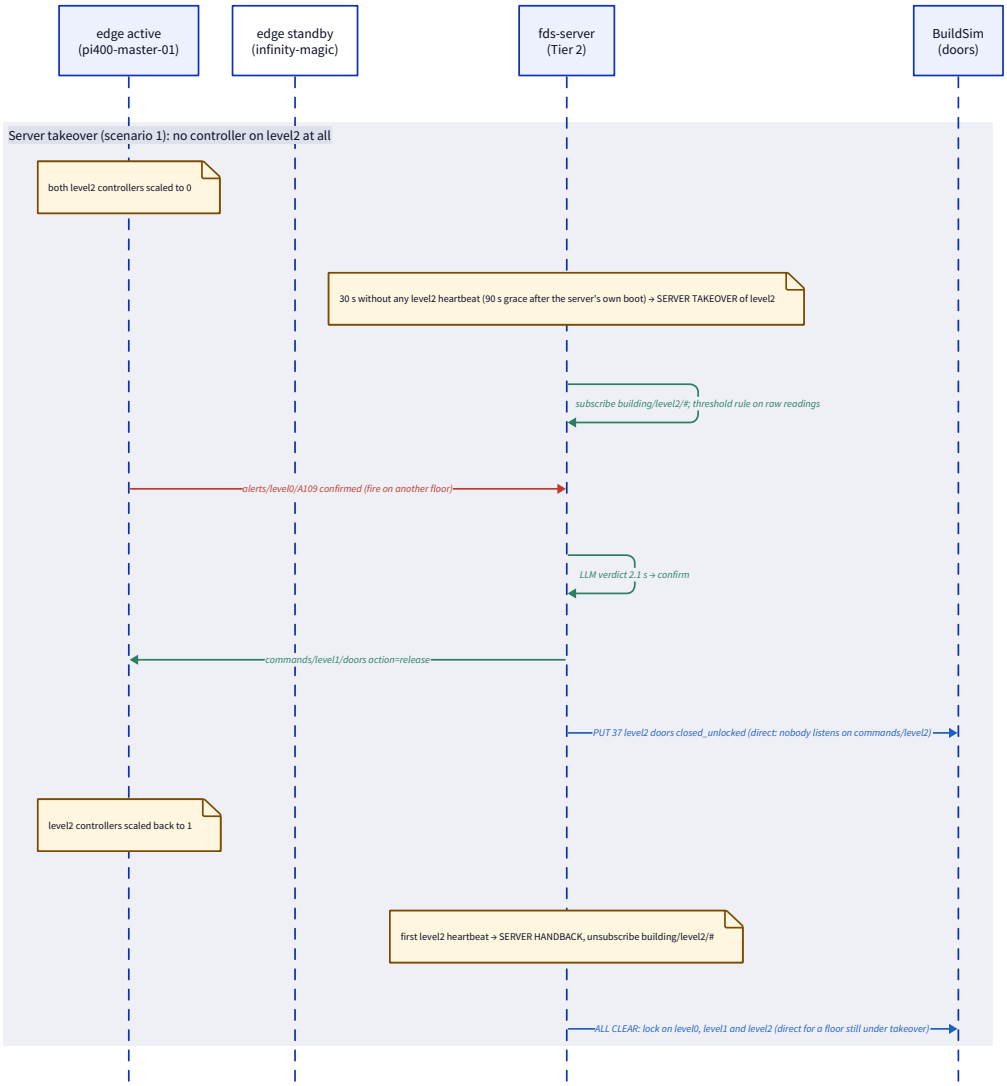
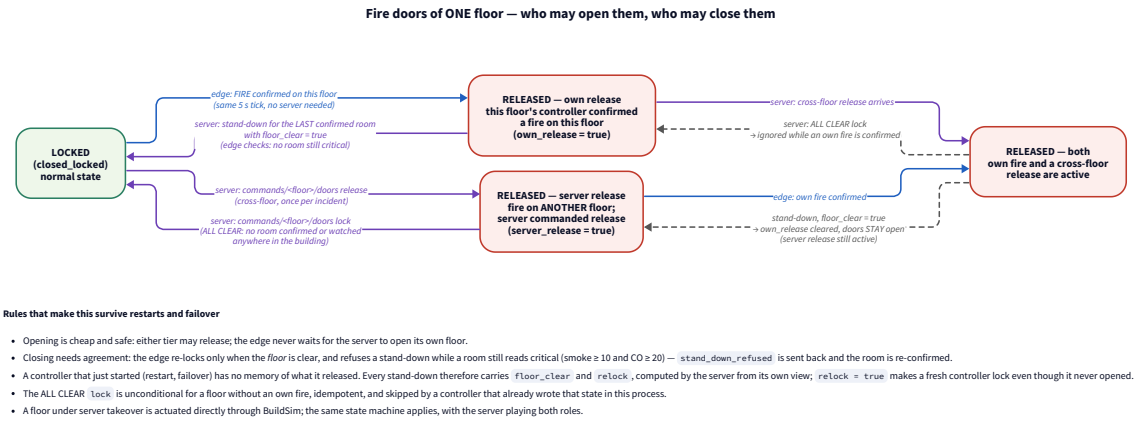


Figure 7: Scenario 1: no controller on level2; the server covers the floor.

8.2 Door lifecycle (state diagram)

The fire doors of one floor are the state that matters, so the state diagram is theirs rather than a process's (Figure 8). Opening is cheap and safe, so either tier may release. Closing needs agreement: the edge re-locks only when its floor is clear, and refuses a stand-down while a room still reads critical. Because a controller that just started has no memory of what it released, every stand-down carries `floor_clear` and `relock` computed from the server's view, and the ALL CLEAR lock is unconditional for a floor without an own fire. A floor under server takeover follows the same machine with the server in both roles.



Rules that make this survive restarts and failover

- Opening is cheap and safe: either tier may release; the edge never waits for the server to open its own floor.
- Closing needs agreement: the edge re-locks only when the *floor* is clear, and refuses a stand-down while a room still reads critical (smoke ≥ 10 and CO ≥ 20) — `stand_down_refused` is sent back and the room is re-confirmed.
- A controller that just started (restart, failover) has no memory of what it released. Every stand-down therefore carries `floor_clear` and `relock`, computed by the server from its own view; `relock = true` makes a fresh controller lock even though it never opened.
- The ALL CLEAR `lock` is unconditional for a floor without an own fire, idempotent, and skipped by a controller that already wrote that state in this process.
- A floor under server takeover is actuated directly through BuildSim; the same state machine applies, with the server playing both roles.

Figure 8: Door states of one floor, who may move between them, and the rules that make the machine survive restarts and failover.

Each process also has a small lifecycle: a controller starts, discovers its doors (retrying until BuildSim answers), connects to the broker (retrying with backoff), runs its tick, heartbeat, peer-watch and command loops, and on SIGTERM cancels them and exits 0, which is why pods stop as `Completed`.

9. Deployment and component placement

The system runs in namespace `ai-edge-intel-course` of a k3s cluster (Figure 2); the placement is the edge–cloud continuum the course is about.

Node	Hardware	What runs there and why
pi400-master-01	Raspberry Pi 400, ARM64, 4 GB	The three active edge controllers (150 m CPU / 192 Mi requested each). This is where a fire panel would sit: detection and door release with no dependency on the rest of the cluster.
infinity-magic	amd64 worker	The three standby controllers, same image and spec. A different node so a node failure cannot take both controllers of a floor.
proart-master-01	Ryzen 9 9950X, 128 GB, RTX 5060 Ti 16 GB	Control plane; `fds-server` (physics, agent, pipeline, control API); BuildSim; the LLM (SGLang behind LiteLLM, namespace `ai-inference`); Mosquitto and TimescaleDB (`data-infra`); Grafana, Phoenix, ClearML, Vault.

Images are built on the control plane with `buildx` for arm64 and amd64, shipped to each node and imported into containerd (`imagePullPolicy: Never`); `scripts/deploy.sh` does the build, ship, apply and restart, and `deploy.sh check` is the pre-flight: images present on each node, the application Secret present, pods healthy. The pre-flight exists because a Pi reboot once garbage-collected the edge image and the actives came up `ErrImageNeverPull`. Secrets reach the server pod through the Kubernetes Secret `fds-app`; no Vault token exists in the namespace. The documented start order after a full stop is BuildSim, then the server and the six controllers together; the server's 90 s boot grace exists because starting it 45 s before the controllers once produced a takeover of every floor, harmless but alarming.

An ArgoCD application for the manifests was set up and then removed: the images never leave the cluster so GitOps could not deploy them, and its `selfHeal` would have reverted the scale-to-zero the failover tests rely on. The decision is recorded in Section 4.4.

10. Test plan

Three layers. **Unit tests** run offline with the broker, BuildSim, the ONNX session, the LLM and the database all faked: 79 tests, covering the Tier-1 controller (decision rule, alert gate, door ownership, failover, broker retry, stale sensors), the Tier-2 policy (dedupe, verdict gate, watch, stand-down, ALL CLEAR, takeover, viewer cards), graceful degradation (fallback with the LLM unreachable) and the physics, sensor, detector, spread, evacuation and occupancy models. CI runs `ruff` and the suite on every push. **Acceptance scenarios** run against the live cluster with the viewer and the dashboards open; each has a pass criterion checked in the server log, the controller logs, the database and BuildSim's door states (`scripts/door_check.py`). **Data checks** (`scripts/db_check.py`) count what the run left behind.

ID	Scenario	Requirements	Pass criterion
T-01	Scale both level2 controllers to 0; fire on level0; restore level2	SAF-03, SAF-01	Takeover logged ≈30 s after scale-down; level2 doors released directly; hand-back on first heartbeat; ALL CLEAR re-locks all floors
T-02	Scale LiteLLM to 0; fire on level1; restore	NFR-02, FR-03	Fallback confirms within one tick; cross-floor release; cards in fallback mode; verdicts resume after restore
T-03	Restart Mosquitto; fire afterwards	NFR-06, NFR-07	Edges and server reconnect; no takeover; fire handled with real verdicts
T-04	Cold start of the namespace in the documented order	NFR-01	Doors discovered on every floor; six heartbeats; no takeover; no spurious alert for 2 min; 950 rooms in `/status`
T-05	Smoke-only event (`/false-alarm/trigger`)	FR-02, SAF-04	Edge `possible`; LLM rejects; watch; stand-down with `relock=false`; zero door commands
T-06	Baseline fire, 45 s, all systems healthy	FR-01, FR-03, FR-04, FR-05, FR-06, SAF-01, NFR-04	Edge release within 60 s of ignition; verdict; cross-floor release within 1 s; routes; spread to adjacent rooms only; ALL CLEAR
T-07	Fire on level0 and a smoke-only event on level1 within seconds	FR-02, SAF-04	Parallel verdicts; level1 on watch, never escalates on its own; its doors released only by cross-floor, re-locked only at ALL CLEAR
T-08	Delete the active controller pod while its floor is released	SAF-02, SAF-04	Standby promotes ≤30 s; doors stay released; stand-down and re-lock via the controller present; demote on return
T-09	Data and model: row counts, dashboards over 3 h, retrain on the new data	NFR-03, NFR-04	All tables grew; dashboards show every scenario; retrain metrics at or above the deployed model; ONNX parity 1.000
T-10	`ruff`, secret scan, pod spec review	NFR-05	No secrets in the repository; no Vault token in the namespace
T-U	Unit suite	FR-05, FR-06, NFR-07 and all policy rules	79 passed

11. Evaluation and results

The system was evaluated at full building scale against the requirements during a two-day acceptance run on 3–4 October 2026. The point of this section is evidence; every number below comes from a log line, a database query or a test output recorded during that run.

Acceptance scenarios. All nine passed. Scenarios 1 and 2 were run twice because the first run of each found a defect that was fixed and re-verified (below).

T	Result	Measured
T-01	pass (2 runs)	takeover at +30 s; 37 level2 doors released directly; hand-back 20 s after controllers returned
T-02	pass (2 runs)	fallback confirmed the origin room 84 ms after the alert; cross-floor release; cards "(fallback mode)"
T-03	pass	server reconnected in 6 s, controllers in 13 s; no takeover; fire handled with verdicts of 1.9–2.5 s
T-04	pass	318/351/287 rooms, 41/46/37 doors discovered, six heart-beats; start-order takeover seen once, fixed with the boot grace
T-05	pass	rejected in 1.9 s (conf. 0.10); watch; stand-down at +2 min; zero door events
T-06	pass	see the latency table
T-07	pass	two verdicts 1.7 s apart on different floors; level1 stayed on watch; A109 confirmed by the LLM despite a dead CO sensor
T-08	pass	promote 30 s after last heartbeat, demote 5 s later; doors held; final state all locked
T-09	pass	183 M readings / 18 GB; retrain reproduced the deployed model; parity 1.000

Latency (FR-01, FR-03, SAF-01, NFR-04). From scenario 6 and the 48-hour database window around the run:

Step	Measured	Requirement
Ignition → edge `fire` decision (A109)	14.7 s	< 60 s ✓
Edge decision → own-floor doors released	same tick (PUT before the log line)	same tick ✓
Isolation forest per room, Pi 400	p50 17.8 ms, p95 33.0 ms (n=3 035)	p95 < 100 ms ✓
Isolation forest per room, amd64 standby	p50 2.3 ms, p95 6.4 ms (n=3 000)	—
Edge alert → server receipt	≈1 ms	—
LLM verdict (confirm / reject, n=40)	p50 2.1 s, p95 3.2 s	—
Verdict → cross-floor release command	25 ms	< 1 s ✓
Standby promotion after controller loss	30 s	≤ 30 s ✓
Server takeover of a silent floor	30 s	≤ 30 s ✓
Fire out → ALL CLEAR (45 s fire)	7 min 21 s	evidence-based (FR-06) ✓

The verdict latency fell from 5.2 s p50 / 7.0 s p95 with the original full-plan prompt to 2.1 / 3.2 s with the verdict-only contract, and alert→verdict queueing under a cluster of alerts fell from 24 s p50 to nothing once alerts were assessed three at a time.

Detection model (FR-02, NFR-04). The edge model is an isolation forest trained on real operation. The retrain at the end of the acceptance run used 16 000 real normal windows, 36 LLM-confirmed fire windows, 16 LLM-rejected windows and 1 600 augmented nuisance windows:

Metric	Deployed model	Retrained 4 Oct
Real normal rooms not flagged	0.998	0.997
LLM-confirmed fires detected	0.917 (33/36)	0.917
sklearn / ONNX parity	0.999	1.000

The deployed model was kept. For comparison, the synthetic-trained model the project started with flagged 97 % of real normal rooms at scale; moving training to real windows is what made the ambiguous band usable. The three fires the model misses are caught by the deterministic rule, which is the point of not letting the model veto it.

Scale (NFR-01, NFR-03). All 950 rooms (956 published spaces) are monitored; each controller reports its floor's count every tick. Ingestion sustained ≈410 readings/s with the consumer queue at zero; a start-up burst of 974 messages drained within a minute. Eight days of operation produced 183 M readings (18 GB), 252 k heartbeats, 11 k edge decisions and 417 Tier-2 events.

What the run found and fixed. Six defects surfaced during acceptance, all fixed, re-tested and committed during the run:

1. **Spread across a courtyard.** A 50-unit centroid radius made A323 a neighbour of A109; every A109 fire lit the far wing. Neighbours are now door-graph edges plus a 30-unit wall radius.
2. **A stuck sensor held the stand-down.** `level0-A108-co` froze at 5.09 ppm for four minutes, just above the 5.0 clear threshold, keeping three floors open until it recovered. The edge now reports sensors frozen for 60 s as stale and the stand-down ignores them (never both smoke and CO).
3. **The fallback doubted the origin room.** With the LLM down, a reading-scaled confidence put a textbook fire at 0.25; the neighbours were confirmed first and the cross-floor release went out 25 s late on the wrong room. The fallback now applies the multi-sensor rule with confidence 0.9.
4. **A dead CO sensor in the fire room blocked detection.** `level0-A109-co` stuck at 0.00; the two-sensor rule could never fire and the LLM read "smoke without CO" as a nuisance; the floor was handled 11 s later through the neighbours. Both tiers now use the stale list: strong smoke alone confirms when CO is stale, and the LLM is told which sensors are faulty.
5. **Start-order takeover.** Starting the server 45 s before the controllers produced a takeover of all floors; a 90 s boot grace fixed it.
6. **Duplicate door writes.** The stand-down's re-lock and ALL CLEAR both locked the same doors; a controller now skips a state it already wrote.

Honest critique. The LLM's confidence is not calibrated: in scenario 8 it rejected an edge-confirmed room at confidence 0.00 on borderline readings, and the watch rule had to escalate it on readings a minute later. The design absorbs this, but it means the 0.7 gate is a tuning constant rather than a probability. The ignition-to-release time of 14.7 s is dominated by the physics ramp and the 5 s tick, not by inference; a 1 s tick would cost CPU on the Pi for little benefit. The seven-minute ALL CLEAR is the CO decay model; it is correct behaviour but would feel slow in a demo. Finally, the acceptance run was performed by the author against a simulator the author also wrote; the fault models are plausible but not validated against real sensors.

[Insert: `figures/buildsim-f0-fire.png`]

Figure 9: BuildSim viewer during scenario 6: fire effects and cards on A109 and its neighbours, doors released.

[Insert: `figures/grafana-operations-3h.png`]

Figure 10: Operations dashboard over the acceptance afternoon: controllers, door state, incident timeline per room, readings of the rooms Tier 1 flagged, Tier-1 and Tier-2 event rates.

12. Dashboard

Two Grafana dashboards are built from code (`grafana/build_dashboards.py`) and provisioned by script, so they are reproducible and reviewable in git. Both share a floor filter.

Operations dashboard (for the operator):

- **Stat row:** edge controllers alive (of 6), confirmed fires, rooms on watch, floors with doors released, floors under server takeover, sensor feed (rooms/min). Green when everything is normal; the design principle is **green row and empty charts means the building is safe**.
- **Edge controllers** table: level, role, node, alive, acting; a promoted standby shows `ACTING`.
- **Door state per floor** with the time of the last change.
- **Incident timeline**: one row per room with a Tier-2 event, coloured by state (confirmed, watching, escalated, stood down).
- **Smoke, CO and temperature** of the rooms Tier 1 flagged, so the charts follow the incident rather than a fixed room.
- **Tier-1 decisions and Tier-2 verdicts per 30 s**, the incident log with reasons, and the edge alert table.

Technical dashboard (for engineers): isolation-forest latency per node, alert→verdict latency, LLM inference p50/p95 and per call, heartbeats per controller, failover and takeover events, verdict mix, pipeline throughput and table sizes.

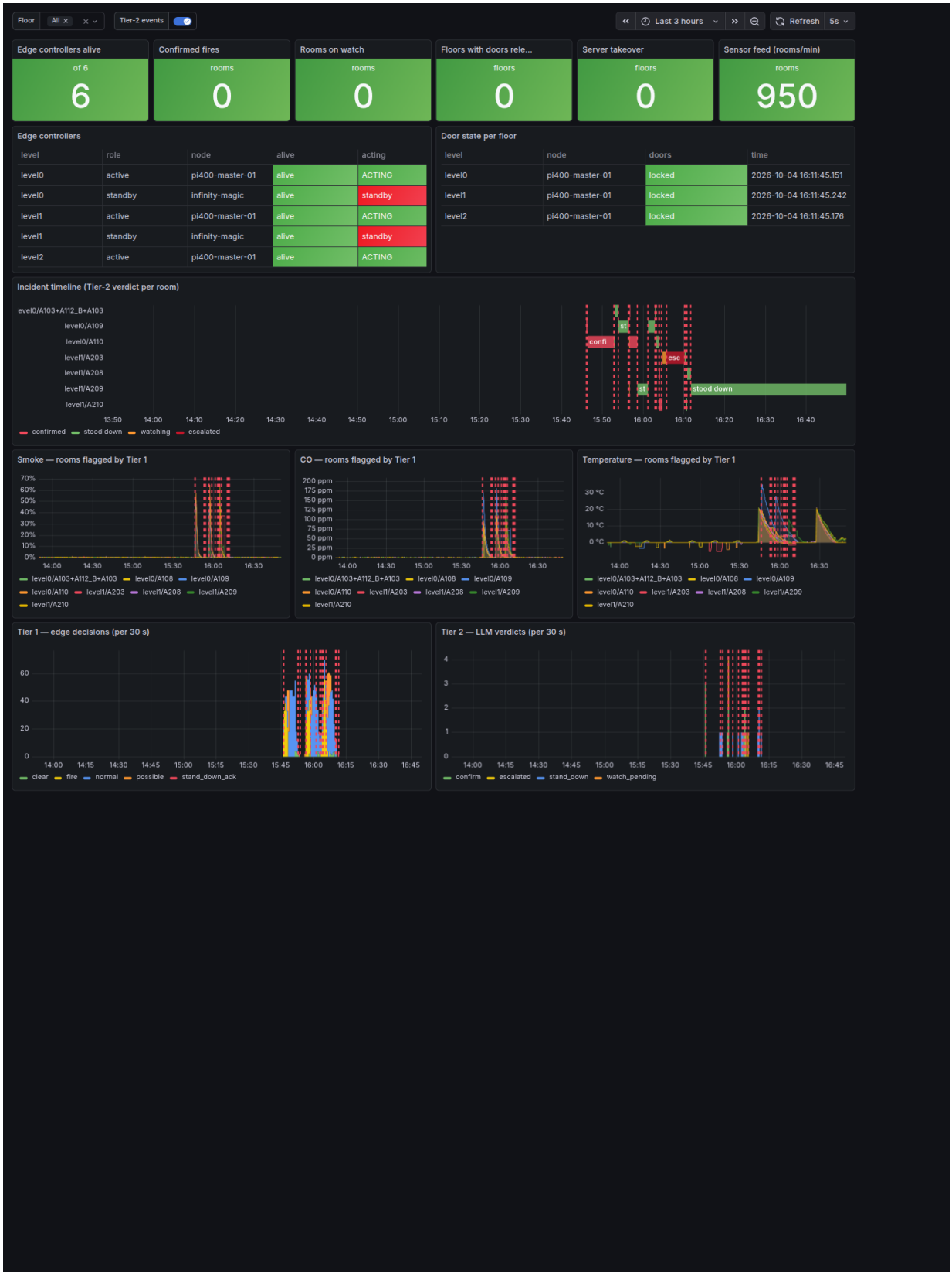


Figure 11: Technical dashboard over the same afternoon.

The BuildSim viewer is the third display and the one that makes the behaviour visible to a non-engineer: fire effects, red cards for confirmed rooms, orange “under observation” cards for rooms the LLM doubted, door states, and everything cleared at ALL CLEAR.

13. Risks and critical reflection

A server restart in the middle of an incident loses the server's state. The controllers keep their floors released and nothing is lost on the life-safety side, but the new server does not know an incident is open and no stand-down would ever arrive. During the run this did not bite because ALL CLEAR had already gone out before each redeploy. The mitigation is designed, an edge-side self-clear after a long clear window and a door-state recovery at server boot, and not built. This is the most important known limitation.

The LLM's confidence is a number, not a probability. It works as a gate because readings decide in the end, but a model built to emit calibrated decisions would fit the verdict function better. "System One" decision models (Jev, and open-weight equivalents such as OpenJev and Laya) appeared during the project; they return typed answers with probabilities in a few hundred milliseconds. The architecture is already model-agnostic at the verdict boundary, so the natural next step is a hybrid Tier 2: a calibrated decision model for the verdict on the decision path, the LLM kept for reasoning and alert text off it. A fair evaluation would replay the acceptance run's alerts through such a model and compare verdicts and latency with the Qwen results recorded in `llm_overrides`.

Thresholds are hand-set. Smoke 10 / CO 20 for fire, 5 for the ambiguous band, 2 / 5 for clear, 60 s windows, 0.7 confidence. They were chosen from the physics model and tuned on the acceptance run; a real building would need them calibrated per sensor type and installation.

The simulator and the system share an author. Fault models (noise, drift, stuck) are plausible rather than validated, and the physics is optimistic about how cleanly smoke and CO separate a fire from a nuisance. Results are reported against the requirements, not as absolute claims about real buildings.

What would be done with more time:

- Build the restart-mid-incident mitigation.
- The hybrid Tier 2 above, with the offline replay as the evaluation.
- Per-sensor calibration profiles and HVAC as a negative feedback term in the physics.
- Stairwell-aware cross-floor response: release only the doors on affected routes rather than every door on the other floors.
- An operator question channel: "why was A208 confirmed?" answered from the stored reasoning.

Risk / assumption	Impact	Mitigation / decision
LLM unavailable	No contextual judgement	Rule-based fallback, exercised live (T-02)
Active controller dies	Floor undetected	Standby on another node, 30 s (T-08)
Both controllers of a floor gone	Floor undetected	Server takeover with direct door actuation (T-01)
Server restarts mid-incident	Stand-down never sent	Known limitation; edge self-clear + boot recovery designed
Stuck sensor	Detection blocked or stand-down held	Stale-sensor rule in both tiers (T-03, T-U)
Broker restart	Messages lost for seconds	Automatic reconnect; edge keeps ticking (T-03)
LLM confidence uncalibrated	Wrong gate decisions	Readings decide on watch; hybrid Tier 2 as future work
Edge image garbage-collected on the Pi	Actives cannot start	`deploy.sh check` pre-flight before every start
Simulated physics optimistic	Detection times differ in real buildings	Reported relative to requirements, not absolute